第10章 关系查询处理和查询优化

关系数据库的一个重要特点是存储结构和存取路径对用户隐蔽，用户访问数据库时，只需要用SQL语句表达自己需要哪些信息即可，而不需要像过程化语言那样具体指定怎么去做。将用户的查询请求转化成查询执行计划并高效地执行，是关系数据库管理系统的重要功能。本章将介绍关系数据库管理系统如何处理用户的查询请求，以及如何进行查询优化。

本章导读

关系数据库的查询处理和查询优化技术是数据库管理系统的重要功能，它极大地影响着关系数据库的性能。

本章首先介绍关系数据库管理系统的查询处理(query processing)步骤，然后深入讨论查询优化(query optimization)技术。查询优化一般可分为代数优化(也称为逻辑优化)和物理优化(也称为非代数优化)。代数优化是指关系代数表达式的优化，物理优化则是指通过存取路径和底层操作算法选择进行的优化。本章讲解实现查询操作的主要算法思想，目的是使读者**初步了解关系数据库管理系统查询处理的基本步骤，以及查询优化的概念、基本方法和技术**，为数据库应用开发中利用查询优化技术提高查询效率和系统性能打下基础。

学习完本章内容，要了解关系查询处理的常用算法，理解为什么关系数据库要进行查询优化，掌握查询优化的几种方法，其中重点和难点是深入理解基于代价估算的关系查询物理优化策略。

10.1 关系数据库管理系统的查询处理

查询处理是关系数据库管理系统执行查询语句的过程，其任务是把用户提交的查询语句转换为高效的**查询执行计划**并执行。

10.1.1 查询处理步骤

关系数据库管理系统的查询处理可以分为4个阶段：查询分析、查询检查、查询优化和查

询执行，如图 10.1 所示。

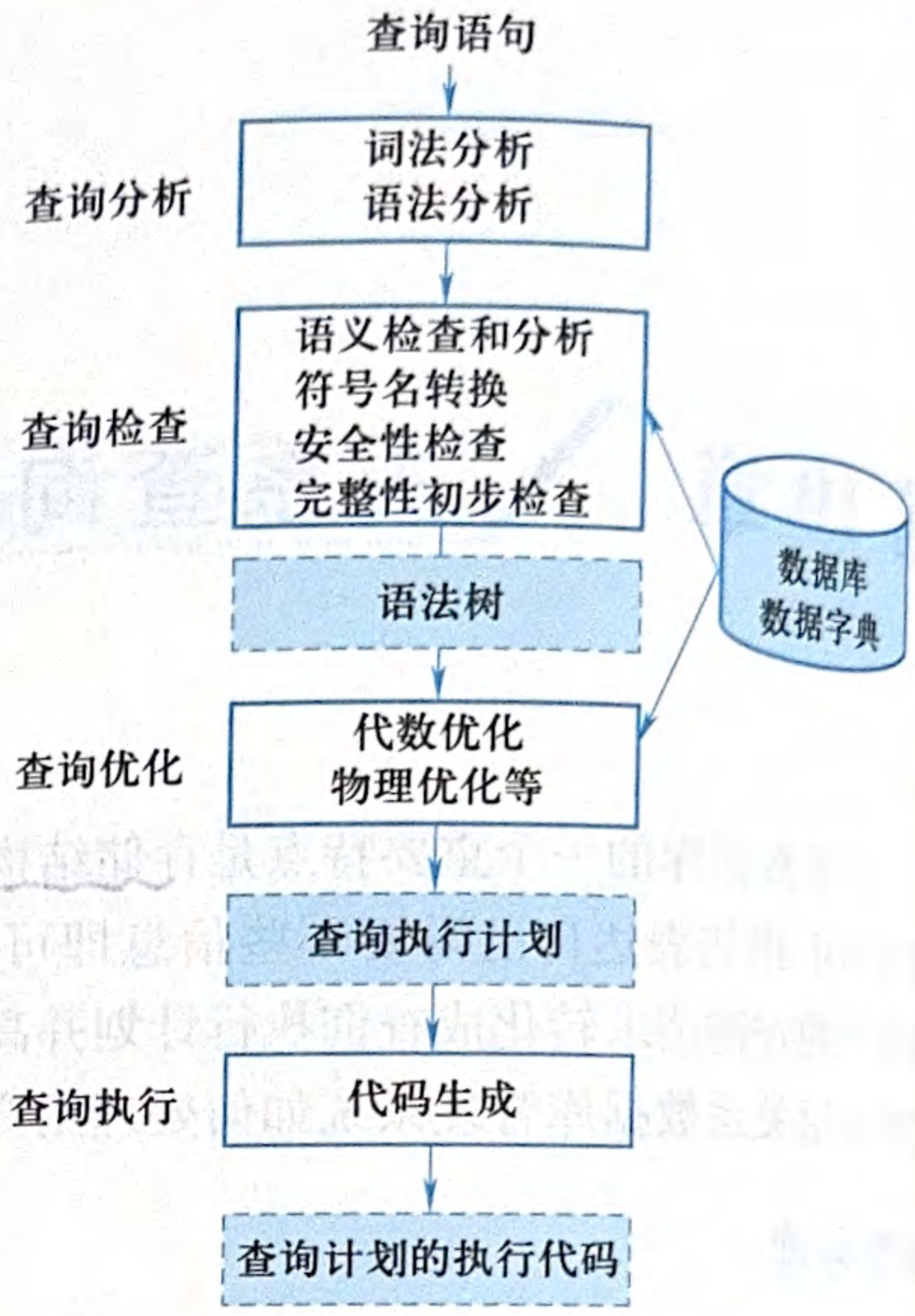


图 10.1 查询处理步骤

1. 查询分析

首先对查询语句进行扫描、词法分析和语法分析。从查询语句中识别出语言符号，如 SQL 关键字、属性名和关系名等，进行语法检查和语法分析，即判断查询语句是否符合 SQL 语法规则。如果没有语法错误就转入下一步处理，否则便报告语句中出现的语法错误。

2. 查询检查

对合法的查询语句进行语义检查和分析，即根据数据字典中有关的模式定义，检查语句中的数据库对象(如关系名、属性名)是否存在和有效。如果是对视图的操作，则要用视图消解方法把对视图的操作转换成对基本表的操作。还要根据数据字典中的用户权限和完整性约束定义，进行安全性和完整性检查。如果该用户没有相应的访问权限或违反了完整性约束，则拒绝执行该查询。当然，这时的完整性检查是初步的、静态的检查。检查通过后便把 SQL 查询语句转换成内部表示，即等价的**关系代数表达式**。这个过程中要把数据库对象的外部名称转换为内部表示，即符号名转换。关系数据库管理系统一般都用**语法树**(syntax tree)来表示扩展的关系代数表达式。

3. 查询优化

每个查询都会有许多可供选择的执行策略和操作算法，查询优化就是选择一个高效执行的查询处理策略。查询优化有多种方法，按照优化的层次一般可将查询优化分为代数优化和物理优化。**代数优化是指关系代数表达式的优化**，即按照一定的规则，通过对关系代数表达式进行等价变换以改变代数表达式中操作的次序和组合，使查询执行更高效；**物理优化则是指存取路径和底层操作算法的选择**。查询优化方法选择的依据可以是基于规则(rule based)的，也可以是基于代价(cost based)的，还可以是基于语义(semantic based)的。

实际关系数据库管理系统中的查询优化器都综合运用了这些优化技术，以在保证查询优化效率的同时获得较好的查询优化效果。

4. 查询执行

依据查询优化器得到的执行策略生成查询执行计划，由**代码生成器**(code generator)生成执行这个查询计划的代码，然后加以执行，回送查询结果。

10.1.2 实现查询操作的算法

本小节简要介绍查询操作中选择操作和连接操作的实现算法，确切地说是算法思想。每一

种操作可以有多种执行算法，这里仅仅介绍最主要的几个算法，对于其他重要操作的详细实现算法，有兴趣的读者请参考有关关系数据库管理系统实现技术的阅读材料。

1. 选择操作的实现

第3章中已经介绍了 SELECT 语句的强大功能。SELECT 语句有许多选项，因此实现的算法和优化策略也很复杂。不失一般性，下面以简单的选择操作为例介绍典型的实现方法。

[例 10.1]针对如下 SQL 语句：

```
SELECT * FROM Student WHERE <条件表达式>;
```

考虑<条件表达式>的几种情况：

C1：无条件；

C2：Sno='20180003'；

C3：Sbirthdate>='2000-1-1'；

C4：Smajor='计算机科学与技术' AND Sbirthdate>='2000-1-1'；

选择操作只涉及一个关系，一般采用全表扫描(full table scan)算法或索引扫描(index scan)算法。

(1) 全表扫描算法

假设可以使用的内存为 M 块，全表扫描选择算法思想如下：

① 按照物理次序读 Student 表的 M 块到内存。

② 检查内存的每个元组 t，如果 t 满足选择条件，则输出 t。

③ 如果 Student 表还有其他块未被处理，则重复①和②。

全表扫描算法只需要很少的内存(最少为 1 块)就可以运行，而且控制简单，对于规模小的表，这种算法简单有效。对于规模大的表进行顺序扫描，当选择率(即满足条件的元组数占全表的比例)较低时，这个算法效率很低。

(2) 索引扫描算法

对于规模较大的表，如果选择条件中的属性上有索引，则可以采用索引扫描算法。通过索引先找到满足条件的元组指针，再通过元组指针在查询的基本表中找到相应元组。

[例 10.1-C2]以例 10.1 中的 C2 为例。

```
Sno='20180003'
```

如果 Sno 上有索引，则可以使用索引得到 Sno 为'20180003'的元组的指针(逻辑地址)，然后通过元组指针在 Student 表中检索到该学生。

[例 10.1-C3]以例 10.1 中的 C3 为例。

```
Sbirthdate>='2000-1-1'
```

如果 Sbirthdate 上有 $B+$树索引，则可以使用 $B+$树索引找到 Sbirthdate>='2000-1-1'的第一

个索引项，以此为入口点，在 $B+$树的顺序集上得到满足该条件的所有元组指针，然后通过这些元组指针到 Student 表中检索到所有 2000 年 1 月 1 日以后出生的学生。

[例 10.1-C4]以例 10.1 中的 C4 为例。

```
Smajor='计算机科学与技术' AND Sbirthdate>='2000-1-1'
```

如果 Smajor 和 Sbirthdate 上都有索引，一种算法是：分别用上面两种方法找到 Smajor='计算机科学与技术'的一组元组指针和 Sbirthdate>='2000-1-1'的另一组元组指针，求这两组指针的交集，再到 Student 表中检索，即可得到主修专业为计算机科学与技术且在 2000 年 1 月 1 日以后出生的学生。

另一种算法是：找到 Smajor='计算机科学与技术'的一组元组指针，通过这些元组指针到 Student 表中检索，并对得到的元组检查另一个选择条件 Sbirthdate>='2000-1-1'是否满足，然后把满足条件的元组作为结果输出。

一般情况下，当选择率较低时，索引扫描算法要优于全表扫描算法。但在某些情况下，例如选择率较高，或者要查找的元组均匀地分布在查找的表中，这时索引扫描算法的性能则不如全表扫描算法。因为除了对表的扫描操作，还要加上对 $B+$树索引的扫描操作，对每一个检索码，从 $B+$树根结点到叶结点路径上的每个结点都要执行一次 I/O 操作。

2. 连接操作的实现

连接操作是查询处理中最常用也是最耗时的操作之一。人们对它进行了深入的研究，提出了一系列算法。不失一般性，这里通过例子简单介绍等值连接(或自然连接)最常用的几种算法思想。

[例 10.2]针对如下 SQL 语句：

```
SELECT * FROM Student,SC WHERE Student.Sno=SC.Sno;
```

(1) 嵌套循环连接(nested loop join)算法

嵌套循环连接算法由两层循环组成，其基本思想是：对外层循环(Student 表)的每一个元组，依次检索内层循环(SC 表)中的每一个元组，并检查这两个元组在连接属性(Sno)上是否相等。如果满足连接条件，则串接后作为结果输出，直到外层循环表中的元组处理完为止。

前面已经介绍过，物理块是存储分配和 I/O 操作的基本单位，也就是说，表中的数据实际上是以物理块为单位进行存取。假设共有 M 块可用内存，基于块的嵌套循环连接算法的步骤如下：

① 将 Student 表的 $M-1$ 块数据读到内存。

② 读入 SC 表的 1 块数据。

③ 对 Student 表在内存中的每一个元组，检索 SC 表在内存中的每一个元组，如果满足连接条件，则串接后作为结果输出。

④ 继续读入 SC 表的下一个数据块，重复第③步，直到 SC 表处理一遍。

⑤ 继续读取 $M-1$ 块 Student 表数据，重复第②~④步，直到 Student 表处理完毕。

嵌套循环连接算法是最简单且最通用的连接算法，可以处理包括非等值连接在内的各种连接操作。

(2) 排序-合并连接(sort-merge join)算法

排序-合并连接算法是等值连接常用的算法，尤其适合参与连接的诸表已经按连接属性排好序的情况。该算法的步骤如下：

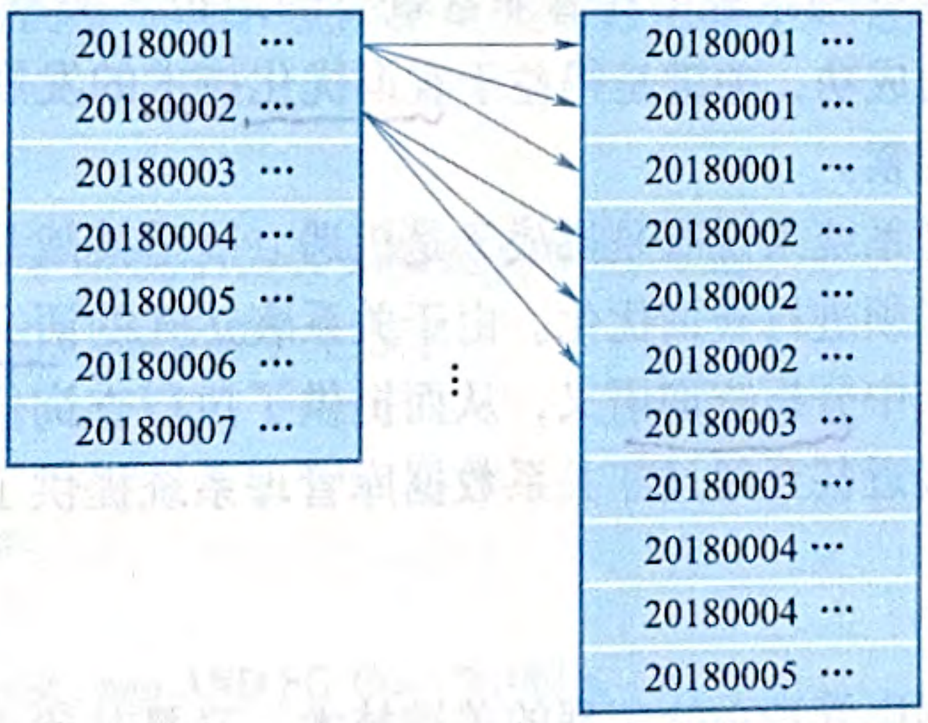
① 如果参与连接的 Student 表和 SC 表没有排好序，首先分别对这两个表按连接属性 Sno 排序。

② 将排好序的 Student 表和 SC 表中的数据块读到内存中。

③ 依次扫描 SC 表中与 Student 表具有相同 Sno 值的元组，把它们连接起来(如图 10.2 所示)。

④ 当扫描到 Sno 不相同的第一个 SC 元组时，返回 Student 表并扫描它的下一个元组，再扫描 SC 表中具有相同 Sno 的元组，把它们连接起来。

⑤ 在内存中的 Student 表或 SC 表扫描完后，继续将相应表余下的数据块读入内存，重复第③和第④步，直到 Student 表或 SC 表全部扫描完。



类似于归并排序中的归并算法.

图 10.2 排序-合并连接算法示意图

这样 Student 表和 SC 表都只要扫描一遍即可。当然，如果两个表原来无序，执行时间要加上对两个表的排序时间。一般来说，对于大表，即使加上排序时间，使用排序-合并连接算法的总执行时间一般仍会减少。

(3) 索引连接(index join)算法

索引连接算法的步骤如下：

① 在 SC 表上建立属性 Sno 的索引，若已有 Sno 上的索引，则跳过本步骤。

② 对 Student 中每一个元组，由 Sno 值通过 SC 的索引查找相应的 SC 元组。

③ 把这些 SC 元组和该 Student 元组连接起来。

循环执行步骤②和③，直到 Student 表中的元组处理完为止。

(4) 哈希连接(hash join)算法

哈希连接算法把连接属性作为哈希码，用同一个哈希函数对 Student 表和 SC 表中的元组计算哈希值。具体步骤如下：

① 划分阶段，也称为创建阶段，即创建哈希表。对包含较少元组的表(如 Student 表)进行一遍处理，把它的元组按哈希函数(哈希码是连接属性)分散到哈希表的桶中。

② 试探阶段，也称为连接阶段。对另一个表(SC 表)进行一遍处理，把 SC 表的元组也按同一个哈希函数(哈希码是连接属性)进行散列，映射到相应的 Student 表哈希桶，检索该桶中的 Student 表元组，将 SC 表元组与该桶中与之相匹配的 Student 表元组连接起来。

哈希连接算法假设两个表中较小的表在第一阶段(划分阶段)后可以完全放入内存的哈希桶中。如果不满足这个前提条件，则需要对两个表进行分解，相应的哈希连接算法请参考本章文献[16]。

10.2 关系数据库管理系统的查询优化

查询优化在关系数据库管理系统中有着非常重要的地位。关系数据库系统和非过程化的 SQL 之所以能够取得巨大的成功，关键是得益于查询优化技术的发展。查询优化是影响关系数据库管理系统性能的主要因素。

优化对关系数据库管理系统来说既是挑战又是机遇。所谓挑战是指关系数据库管理系统为了达到用户可接受的性能必须进行查询优化。由于关系表达式的语义级别很高，使关系数据库管理系统可以从关系表达式中分析查询语义，从而提供了执行查询优化的可能性。这就为关系数据库管理系统在性能上接近甚至超过非关系数据库管理系统提供了机遇。

10.2.1 查询优化概述

查询优化既是关系数据库管理系统实现的关键技术，又是其优点所在。它减轻了用户选择存取路径的负担。用户只要提出“干什么”，而不必指出“怎么干”。对比一下非关系数据库管理系统中的情况：用户使用过程化的语言表达查询要求，至于执行何种记录级的操作以及操作的序列，是由用户而不是由系统来决定的。因此用户必须了解存取路径，系统要提供给用户选择存取路径的手段，查询效率由用户的存取策略决定。如果用户做了不当的选择，系统是无法对此加以改进的。这就要求用户有较高的数据库技术和程序设计水平。

查询优化的优点不仅在于用户不必考虑如何最好地表达查询以获得较高的效率，而且在于系统可以比用户程序的“优化”做得更好。这是因为：

① 优化器可以从数据字典中获取许多统计信息，例如每个关系表中的元组数、关系中每个属性值的分布情况、哪些属性上已经建立了索引等。优化器可以根据这些信息做出正确的估算，选择高效的执行计划，而用户程序则难以获得这些信息。

② 如果数据库的物理统计信息改变了，系统可以自动对查询进行重新优化以选择相适应的执行计划。在非关系数据库管理系统中则必须重写程序，而重写程序在实际应用中往往是不太可能的。

③ 优化器可以考虑数百种甚至数千种不同的执行计划，而程序员一般只能考虑有限的几种可能性。

④ 优化器中包含很多复杂的优化技术，这些优化技术往往只有最好的程序员才能掌握。系统的自动优化相当于使得所有人都拥有这些优化技术。

关系数据库管理系统通过某种代价模型计算出各种查询执行策略的执行代价，然后选取代价最小的执行方案。在集中式数据库中，查询执行开销主要包括磁盘存取块数(I/O 代价)、处理机时间(CPU 代价)以及查询的内存代价。在分布式数据库中还要加上通信代价，即

$$\text{总代价}=\text{I/O 代价}+\text{CPU 代价}+\text{内存代价}+\text{通信代价}$$

由于磁盘 I/O 操作涉及机械动作，需要的时间与内存操作相比要高几个数量级，因此在计算查询代价时一般用查询处理读写的块数作为衡量单位。

查询优化的总目标是：选择有效的策略，求得给定关系表达式的值，使得查询代价较小。因为查询优化的搜索空间有时非常大，实际系统选择的策略不一定是最优的，而是较优的。

10.2.2 一个实例

首先通过一个简单的例子来说明为什么要进行查询优化。

[例 10.3]求选修了 81003 课程的学生姓名。

用 SQL 语句表达如下：

```
SELECT Student. Sname
FROM Student,SC
WHERE Student. Sno=SC. Sno AND SC. Cno='81003';
```

假定“学生选课管理”数据库中有 1 000 个学生记录，10 000 个选课记录，其中选修 81003 课程的选课记录为 50 个。

系统可以用多种等价的关系代数表达式来完成这一查询，但分析下面三种情况就足以说明问题：

$$Q_1=\Pi_{\text{Sname}}(\sigma_{\text{Student. Sno=SC. Sno}\wedge\text{SC. Cno='81003'}}(\text{Student}\times\text{SC}))$$

$$Q_2=\Pi_{\text{Sname}}(\sigma_{\text{SC. Cno='81003'}}(\text{Student}\bowtie\text{SC}))$$

$$Q_3=\Pi_{\text{Sname}}(\text{Student}\bowtie\sigma_{\text{SC. Cno='81003'}}(\text{SC}))$$

后面将看到由于查询执行的策略不同，查询效率相差很大。

1. 第一种情况

① 计算广义笛卡儿积。把 Student 表和 SC 表的每个元组连接起来。其常用算法是：先在

内存中尽可能多地装入某个表(如 Student 表)的若干块，留出一块存放另一个表(如 SC 表)的元组，然后把读入的 SC 表中的每个元组与内存中的 Student 表的每个元组连接，连接后的元组装满一块后就写到中间文件上，再从 SC 表中读入一块与内存中的 Student 表的元组连接，直到 SC 表处理完；这时再一次读入若干块 Student 元组，读入一块 SC 元组，重复上述处理过程，直到把 Student 表处理完。

设一个数据块能装 10 个 Student 元组或 100 个 SC 元组，在内存中存放 5 块 Student 元组和 1 块 SC 元组，则读取总块数为

$$\frac{1\,000}{10}+\frac{1\,000}{10\times5}\times\frac{10\,000}{100}=100+20\times100=2\,100\text{ 块}$$

其中，读 Student 表 100 块，读 SC 表 20 遍，每遍 100 块，则总计要读取 2 100 个数据块。

连接后的元组数为 $10^3\times10^4=10^7$。设每块能装 10 个元组，则写出 10^6 块。

② 做选择操作。依次读入连接后的元组，按照选择条件选取满足要求的记录。假定内存处理时间忽略。这一步读取中间文件的数量(同写中间文件一样)是 10^6 块。若满足条件的元组假设仅 50 个，则均可放在内存。

③ 做投影操作。把第②步的结果在 Sname 上做投影输出，得到最终结果。

因此第一种情况下执行查询的**总读写数据块=2 100+10^6+10^6**。

2. 第二种情况

① 计算自然连接。为了执行自然连接，读取 Student 表和 SC 表的策略不变，总的读取块数仍为 2 100 块。但自然连接的结果比第一种情况大大减少，连接后的元组数为 10^4 个元组，写出数据块=10^3 块。

② 读取中间文件块，执行选择操作，读取的数据块=10^3 块。

③ 把第②步结果投影输出。

第二种情况下执行查询的**总读写数据块=2 100+10^3+10^3**。其执行代价大约是第一种情况的 1/488。

3. 第三种情况

① 先对 SC 表做选择操作，只需读一遍 SC 表，存取块数为 100 块，因为满足条件的元组仅 50 个，不必使用中间文件。

② 读取 Student 表，把读入的 Student 元组和内存中的 SC 元组做连接。也只需读一遍 Student 表，共 100 块。

③ 把连接结果投影输出。

第三种情况下执行查询**总的读写数据块=100+100**。其执行代价大约是第一种情况的 1/10 000，是第二种情况的 1/20。

假如 SC 表的 Cno 字段上有索引，第一步就不必读取所有的 SC 元组，而只需读取 Cno='81003'的那些元组(50 个)。存取的索引块和 SC 中满足条件的数据块共 3~4 块。若 Student 表在 Sno 上也有索引，则第二步也不必读取所有的 Student 元组，因为满足条件的 SC 记录仅 50

条，涉及最多50条Student记录，因此读取Student表的块数也可大大减少。

这个简单的例子充分说明了查询优化的必要性，同时也给出一些查询优化方法的初步概念。例如，读者可能已经发现，在第一种情况下连接后的元组可以先不立即写出，而是和第②步的选择操作结合，这样可以省去写出和读入的开销。有选择和连接操作时，可以先做选择操作，例如把上面的代数表达式Q_1、Q_2变换为Q_3，这样参加连接的元组就可以大大减少，这是代数优化。在Q_3中，SC表的选择操作算法可以采用全表扫描或索引扫描，经过初步估算，索引扫描方法较优。同样，对于Student和SC表的连接，利用Student表上的索引，采用索引连接代价也较小，这就是物理优化。

10.3 代数优化

SQL语句经过查询分析和查询检查后变换为语法树，它是关系代数表达式的内部表示。本节介绍基于关系代数等价变换规则的优化方法，即代数优化，也称作逻辑优化。

10.3.1 关系代数表达式等价变换规则

代数优化策略是通过对关系代数表达式的等价变换来提高查询效率。所谓关系代数表达式的等价，是指用相同的关系代替两个表达式中相应的关系所得到的结果是相同的。两个关系表达式E_1和E_2是等价的，可记为$E_1 \equiv E_2$。

下面是常用的等价变换规则，证明从略。

1. 连接运算、笛卡儿积运算的交换律

设E_1和E_2是关系代数表达式，F是连接运算的条件，则有

$$E_1 \times E_2 \equiv E_2 \times E_1$$

$$E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$$

$$E_1 \underset{F}{\bowtie} E_2 \equiv E_2 \underset{F}{\bowtie} E_1$$

2. 连接运算、笛卡儿积运算的结合律

设E_1、E_2、E_3是关系代数表达式，F_1和F_2是连接运算的条件，则有

$$(E_1 \times E_2) \times E_3 \equiv E_1 \times (E_2 \times E_3)$$

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

$$(E_1 \underset{F_1}{\bowtie} E_2) \underset{F_2}{\bowtie} E_3 \equiv E_1 \underset{F_1}{\bowtie} (E_2 \underset{F_2}{\bowtie} E_3)$$

3. 投影运算的串接律

$$\Pi_{A_1,A_2,\cdots,A_n}(\Pi_{B_1,B_2,\cdots,B_m}(E)) \equiv \Pi_{A_1,A_2,\cdots,A_n}(E)$$

其中，E是关系代数表达式，$A_i(i=1,2,\cdots,n)$，$B_j(j=1,2,\cdots,m)$是属性名，且$\{A_1,A_2,\cdots,A_n\}$是$\{B_1,B_2,\cdots,B_m\}$的子集。

4. 选择运算的串接律

$$\sigma_{F_1}(\sigma_{F_2}(E)) \equiv \sigma_{F_1 \wedge F_2}(E)$$

其中，E 是关系代数表达式，F_1、F_2是选择条件。选择运算的串接律说明选择条件可以合并，这样一次就可检查全部条件。

5. 选择运算与投影运算的交换律

$$\sigma_F(\Pi_{A_1,A_2,\cdots,A_n}(E)) \equiv \Pi_{A_1,A_2,\cdots,A_n}(\sigma_F(E))$$

其中，选择条件 F 只涉及属性 $A_1,A_2,\cdots,A_n$。

若 F 中有不属于 $A_1,A_2,\cdots,A_n$的属性 $B_1,B_2,\cdots,B_m$，则有更一般的规则：

$$\Pi_{A_1,A_2,\cdots,A_n}(\sigma_F(E)) \equiv \Pi_{A_1,A_2,\cdots,A_n}(\sigma_F(\Pi_{A_1,A_2,\cdots,A_n,B_1,B_2,\cdots,B_m}(E)))$$

6. 选择运算与笛卡儿积运算的交换律

如果 F 中涉及的属性都是 E_1中的属性，则

$$\sigma_F(E_1 \times E_2) \equiv \sigma_F(E_1) \times E_2$$

如果 $F=F_1 \wedge F_2$，并且 F_1只涉及 E_1中的属性，F_2只涉及 E_2中的属性，则由上面的等价变换规则 1、4、6 可推出：

$$\sigma_F(E_1 \times E_2) \equiv \sigma_{F_1}(E_1) \times \sigma_{F_2}(E_2)$$

若 F_1只涉及 E_1中的属性，F_2涉及 E_1和 E_2两者的属性，则仍有

$$\sigma_F(E_1 \times E_2) \equiv \sigma_{F_2}(\sigma_{F_1}(E_1) \times E_2)$$

它使部分选择在笛卡儿积前先做。

7. 选择运算与并运算的分配律

设 E_1、E_2有相同的属性名，则

$$\sigma_F(E_1 \cup E_2) \equiv \sigma_F(E_1) \cup \sigma_F(E_2)$$

8. 选择运算与差运算的分配律

若 E_1与 E_2有相同的属性名，则

$$\sigma_F(E_1 - E_2) \equiv \sigma_F(E_1) - \sigma_F(E_2)$$

9. 选择运算对自然连接运算的分配律

$$\sigma_F(E_1 \bowtie E_2) \equiv \sigma_F(E_1) \bowtie \sigma_F(E_2)$$

F 只涉及 E_1与 E_2的公共属性。

10. 投影运算与笛卡儿积运算的分配律

设 E_1和 E_2是两个关系表达式，$A_1,A_2,\cdots,A_n$是 E_1的属性，$B_1,B_2,\cdots,B_m$是 E_2的属性，则

$$\Pi_{A_1,A_2,\cdots,A_n,B_1,B_2,\cdots,B_m}(E_1 \times E_2) \equiv \Pi_{A_1,A_2,\cdots,A_n}(E_1) \times \Pi_{B_1,B_2,\cdots,B_m}(E_2)$$

11. 投影运算与并运算的分配律

设 E_1和 E_2有相同的属性名，则

$$\Pi_{A_1,A_2,\cdots,A_n}(E_1 \cup E_2) \equiv \Pi_{A_1,A_2,\cdots,A_n}(E_1) \cup \Pi_{A_1,A_2,\cdots,A_n}(E_2)$$

10.3.2 语法树的启发式优化

本小节讨论应用启发式规则(heuristic rule)的代数优化，这是对关系代数表达式的语法树

进行优化的方法。**典型的启发式规则有：**

① 选择运算应尽可能先做。在优化策略中这是最重要、最基本的一条。它常常可使执行代价节约几个数量级，因为选择运算一般使计算的中间结果大大变小。

② 投影运算和选择运算应同时进行。如有若干投影和选择运算，并且它们都对同一个关系操作，则可以在扫描此关系的同时完成所有这些运算，以避免重复扫描关系。

③ 把投影同其前或其后的双目运算结合起来，没有必要为了去掉某些字段而扫描一遍关系。

④ 把某些选择与在它前面要执行的笛卡儿积结合起来成为一个连接运算，连接(特别是等值连接)运算要比同样关系上的笛卡儿积省很多时间。

⑤ 找出公共子表达式。如果这种重复出现的子表达式的结果不是很大的关系，并且从外存中读入这个关系比计算该子表达式的时间少得多，则先计算一次公共子表达式并把结果写入中间文件是合算的。当查询的对象是视图时，定义视图的表达式就是一种公共子表达式。

下面给出遵循这些启发式规则，应用关系代数等价变换规则来优化关系表达式的算法。

算法：关系表达式的优化。

输入：一个关系表达式的语法树。

输出：优化的语法树。

方法：

① 利用等价变换规则 4，把形如 $\sigma_{F_1 \wedge F_2 \wedge \cdots \wedge F_n}(E)$ 的表达式变换为 $\sigma_{F_1}(\sigma_{F_2}(\cdots(\sigma_{F_n}(E))\cdots))$。

② 对每一个选择，利用等价变换规则 4~9，尽可能把它移到树的叶端。

③ 对每一个投影，利用等价变换规则 3、5、10、11 中的一般形式，尽可能把它移向树的叶端。

注意：等价变换规则 3 使一些投影消失，而等价规则 5 把一个投影分裂为两个，其中一个有可能被移向树的叶端。

④ 利用等价变换规则 3~5，把选择和投影的串接合并成单个选择、单个投影或一个选择后跟一个投影，使多个选择或投影能同时执行，或在一次扫描中全部完成。尽管这种变换似乎违背“投影尽可能早做”的原则，但这样做效率更高。

⑤ 把经过上述变换得到的语法树的内结点分组。每一双目运算(×，⋈，∪，-)和它所有的直接祖先为一组(这些直接祖先是单目运算 σ 或 Π)。如果其后代直到叶结点全是单目运算，则也将它们并入该组，但当双目运算是笛卡儿积(×)，而且后面不是与它组成等值连接的选择时，则不能把选择与这个双目运算组成同一组。把这些单目运算单独分为一组。

[例 10.4]给出例 10.3 中 SQL 语句的代数优化。

```
SELECT Student. Sname FROM Student, SC
WHERE Student. Sno=SC. Sno AND SC. Cno='81003';
```

① 把 SQL 语句转换成语法树，如图 10.3 所示。为了使用关系代数表达式的优化法，不妨假设内部表示是关系代数语法树，如图 10.4 所示。

② 对语法树进行优化。利用等价变换规则 4、6 把选择 $\sigma_{SC.Cno='81003'}$ 移到叶端，则图 10.4 所示的语法树便转换成图 10.5 所示优化后的语法树。这就是例 10.3 中 Q_3 的语法树表示。前面已经分析了 Q_3 比 Q_1、Q_2 的查询效率要高得多。

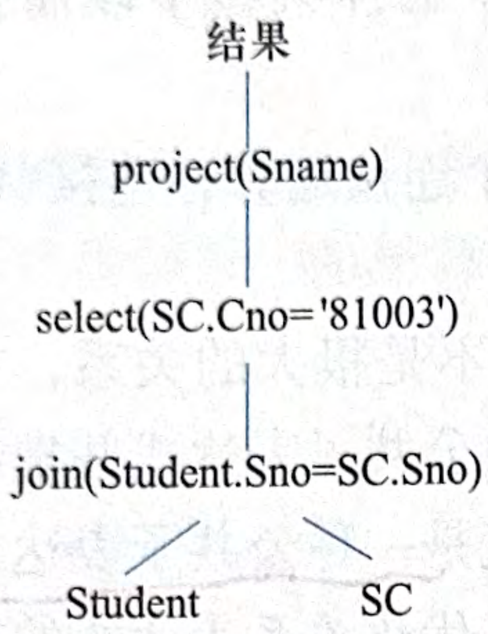


图 10.3 语法树

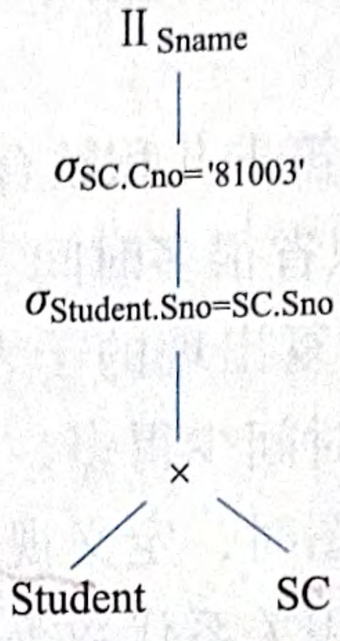


图 10.4 关系代数语法树

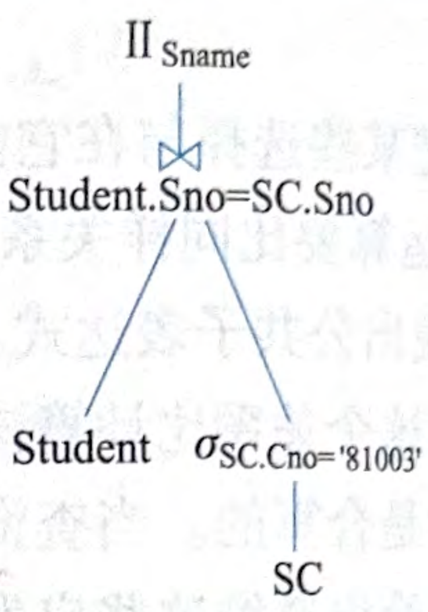


图 10.5 优化后的语法树

10.4 物理优化

代数优化改变查询语句中操作的次序和组合，但不涉及底层的存取路径。10.1.2 小节已讲解了对每一种操作有多种执行这个操作的算法，有多条存取路径，因此对于一个查询语句有多种存取方案，它们的执行效率不同，有的会相差很大。因此，仅仅进行代数优化是不够的。物理优化就是要选择高效合理的操作算法或存取路径，求得优化的查询计划，最终达到查询优化的目标。

选择的方法可以是：

① **基于启发式规则的优化**。启发式规则是指那些在大多数情况下都适用，但不是在每种情况下都是最好的规则。

② **基于代价估算的优化**。使用优化器估算不同执行策略的代价，并选出具有最小代价的执行计划。

③ **两者结合的优化方法**。查询优化器通常会把这两种技术结合在一起使用。因为可能的执行策略很多，要穷尽所有的策略进行代价估算往往是不可行的，会导致查询优化本身付出的代价大于获得的益处。为此，常常先使用启发式规则选取若干较优的候选方案，减少代价估算的工作量，然后分别计算这些候选方案的执行代价，较快地选出最终的优化方案。

10.4.1 基于启发式规则的优化

下面以选择操作和连接操作为例，介绍常用的启发式规则。